

Local Search: (better in space than others).

used for optimization problems. (Where the environment is: fully observable and deterministic.)

①

Hill climbing: best move / accepted only.

↳ gets stuck in local Maxima.

↓ solution.

sideways moves with some probability.

best successor has is current value (so it is not best).

plateaux / Ridges / shoulders.
↳ gets stuck in them.

Variants of HC: HC is incomplete

Steepest Ascent

Random Restart.

starts a series of HC searches with a random initial state.

- each HC has probability of success then the expected number of restarts is $1/p$.

(Very effective)

good for

few local

maxima and

plateaux → finds solution very quickly.

Stochastic Hill climbing

chooses randomly from the best. (uphill moves).

First choice HC:

takes the first best solution he meets.

(best when state has many successors) implements SHC

→ because it generates successors randomly until one is better.

①

II Simulated Annealing: HC + Random Walk.

$\left\{ \begin{array}{l} \xrightarrow{\text{the passes}} \\ \xleftarrow{\text{probability of accepting bad moves}} \end{array} \right.$

$\downarrow$
Efficiency + Completeness

$\star$ Similar to HC: but instead of picking the best move
 - picks a random move.

$t=0$ Temperature ∞
 $\downarrow$ Bad moves
 $t=\infty$ lowest temperature

if better accepted with $P=1$
 if bad accepted with $P < 1$
 and $P \downarrow$ decreases exponentially with the badness of the move ΔE which evaluation is worked.

If T is lowered slowly enough $\rightarrow$ the algorithm will find a global optimum with $p \sim 1$

9

$\Delta E = \text{next.value} - \text{current.value}$
 // next is better $\leftarrow$ if $\Delta E > 0$ then next = current value.
 else: current = next with

Part of the code.

When T is high:
 $P = e^{\frac{\Delta E}{T}} \xrightarrow{T \rightarrow +\infty} 1$
 When T is low:
 $P = e^{\frac{\Delta E}{T}} \xrightarrow{T \rightarrow -\infty} 0$

$$P = e^{\frac{\Delta E}{T}}$$

ΔE is negative and $T \rightarrow \infty \rightarrow 0$

if $\rightarrow 0$ then $e^{-\infty} \rightarrow 0$.
 $\frac{\Delta E}{T} \rightarrow 0$ $\frac{\infty}{\infty} \rightarrow 0$ $e^0 = 1$.

Ⓜ Local Beam Search:

- K randomly generated states
- Searches at the same time.
 - Keeps track of k best many successors of each search. → complete list
 - if one of them is goal returns it halt.

difference from the random restart is that:
threads here are independent.

(Can suffer from a lack of diversity).

*** Stochastic Beam Search: SBS.

- * like stochastic Hk.
- * It chooses k successors at random, with probability of choosing a given one being an increasing function of the value of that successor.

Ⓜ Genetic Algorithm

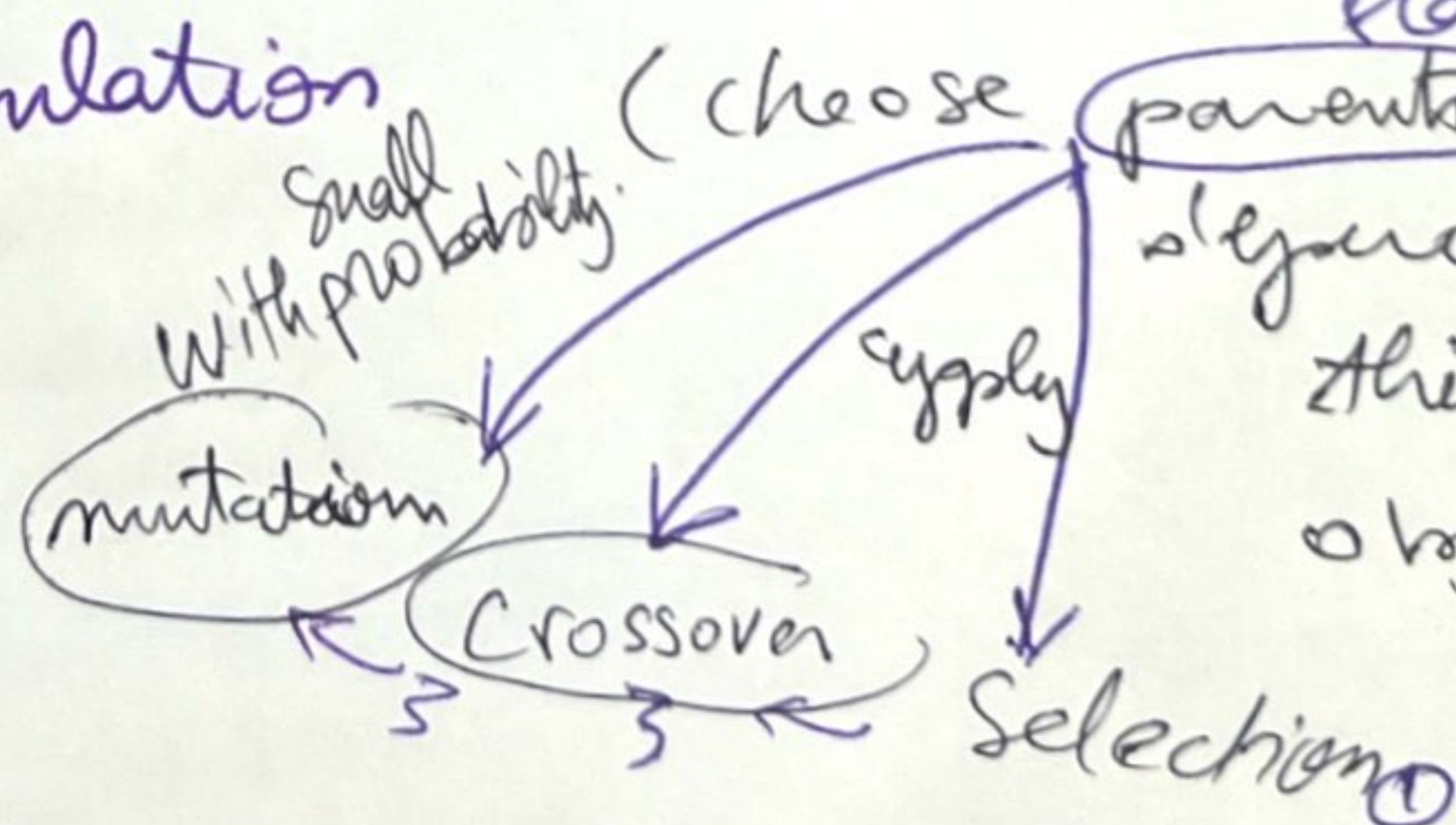
state = individual.

Crossover:

point is chosen randomly from the individuals in parents.

- offspring / children are generated by crossing over strings at the crossover point.

Population



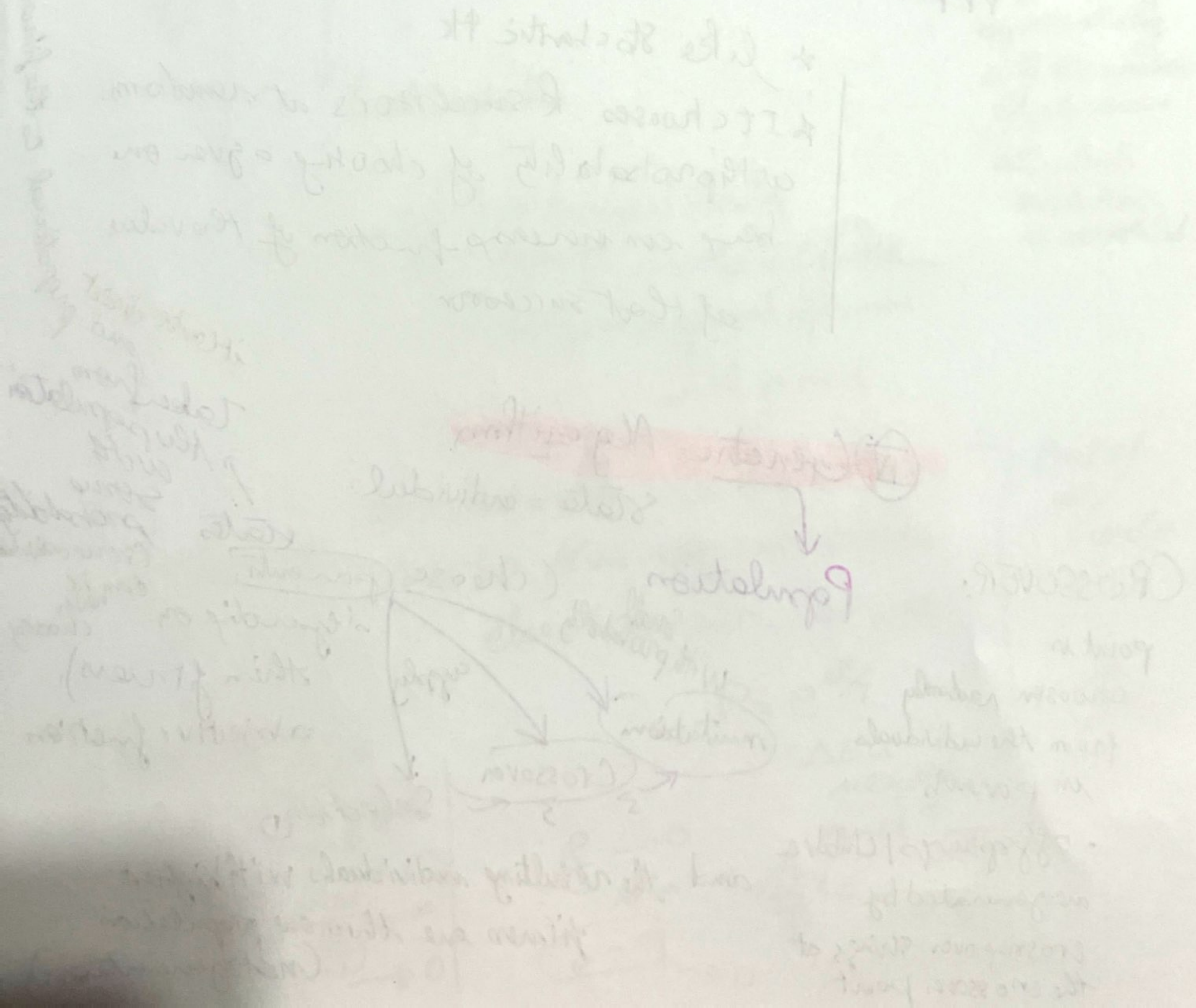
and the resulting individuals with highest fitness are the new population (next generation)

it can be direct and proportional to the fitness score.
taken from the population with some probability (some individuals can be chosen).
states depending on their fitness.
objective function.

Schema:

- Is a substring that is repeated in strings of individuals.
- and strings that match it are called instances of the schema.

→ Result: if the average fitness of the instances of the schema is above the mean then the number of instances of the schema within the population will grow over time.



Called also: Greedy local search

Hillclimbing Versions with some codes:

Random Restart

↳ When we reach a local optimum, we start again from another starting place

↳ When there's only one local optimum and one global
↳ We'll keep getting the same answer

Some Definitions:

Bridges: Sequence of local maxima, that are very difficult to navigate for greedy algorithms.

possible
not to
find solution?

Plateaus: A plateau is a flat area of the state landscape.
↳ Where from which NO uphill exists.
and a **Shoulder** from which progress is possible!

Hillclimbing Versions

Stochastic hill climbing.

↳ Chooses at random from the uphill moves (converges more slowly but better).

First choice hill climbing.

↳ implements Stochastic HC by generating successors randomly until one generated is better than the current state (good when there are many successors).

Random Restart.

↳ Conducts a series of hill-climbing searches from randomly generated initial states until a goal is found.

↳ it is complete with $P(c)=1$ because it may generate the goal state

Which one to choose? at what time??

> few local Maxima and Plateaus:
Random Restart $\rightarrow$ quickly ✓.

①

Code for the Hill Climbing:

each
node
is replaced
by the best
neighbor.

function Hill-climbing (problem) returns a state local Maxima
current = problem.initial-state
while true {
neighbor = highest valued successor state of current.
if $P(\text{neighbor}) > P(\text{current})$ return current
else current = neighbor.

②

Code for Simulated Annealing:

function Simulated-Annealing (problem, schedule) returns
a solution state.
current = problem.initial-state.
for $t = 1$ to ∞ {
T = schedule(t)
if $T = 0$ then return current.
next = random successor of current.
 $\Delta E = h(\text{current}) - h(\text{next})$
if $\Delta E > 0$ then current = next
else current = next with probability $e^{-\Delta E/T}$

next
is better

it means that:
if:
Not 100%!

③ Local Beam Search { keep track of K states rather than only just one state.
↳ select K best from the frontier.
↳ if one is goal then it HALTS.

it has a variant called:

★ Stochastic Beam Search:

↳ helps avoid clustering in a small region of the state space that makes the problem k -times slower than HC.

Instead of choosing the top K successors, Stochastic Beam Search chooses successors with probability proportional to the successor's value (f / or objective function) thus increasing diversity.

Solving 8 Queens Problem, using Hill climbing

Simulated Annealing Code:

```
def simulated-Annealing(state):
```

```
    for t = 1 to ∞ do
```

```
        T ← schedule(t).
```

```
        if T = 0 then
```

```
            return state
```

```
        candidate ← random-neighbor(state)
```

```
        E = candidate - next
```

```
        if E > 0 then
```

```
            state = candidate.
```

```
        else (if it is negative)
```

```
            prob = probability(E, T) =  $e^{\frac{\Delta E}{T}}$ 
```

```
            if (random() < prob) then
```

```
                state ← candidate.
```

to choose the next

We use random method
then compare

$$P(\text{accept-Next}) = e^{\frac{\Delta E}{T}}$$

// accept it

[if random() > prob]
We are not going to
make that move

Loop in python: $T = 100.$ for i in range(T, 1, -1).

{ to talk about the move
it can be simply $\frac{\Delta E}{T}$ }

{ Temperature starts from 100 to down.

Low
function.

Temperature